

Description:

TEC Driver Design – V1.0

We are designing a portable electric cooler for transporting blood samples from hospitals to labs. This system uses a Thermo-Electric Cooler (TEC) to maintain the sample's temperature, based on temperature readings from two redundant NTCs.

Your task is to design a circuit that drives the TEC from 0A to 4A. Your circuit will hook up to a larger system that regulates the sample temperature to 4°C.

TEC Specification:

- 0A to 4A
- 0 to 8V

Please submit your design, with any supporting information you feel would be useful for a Sr. Engineer to review and approve.

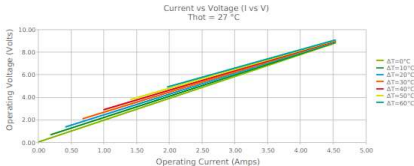
As a bonus, design the larger temperature regulation system.

supposing the mechanical housing for our project will be a sample chamber. The TEC and its circuitry are placed at the bottom like:



The TEC spec ask for 0-4A and 0-8V. I suppose the rating current and voltage to be 4 A and 8 V. I found a product of TEC to be of well match of the spec, and I will use this for reference of Electrical and Thermal Performance within 0A - 4A and 0V to 8V <https://www.digilkey.com/en/products/detail/tark-thermal-solutions/387005514/14113435>

as the data sheet shown,



The I/V relationship tends to be linear, so I will assume the TEC device as an ideal resistor 2 Ω for simulation.

maintain progress at https://github.com/ZDYnb/TEC_Driver

Problem Statement

Base on my understanding, I am to design a driver circuit to drive TEC from 0A to 4 A. The range of the input voltage should be within 0 - 8 V.

Goal is to drive the target load Thermo-Electric Cooler (TEC) electrically safely (within current 0 - 4 A and voltage 0 - 8 V range) and reliably to control the sample temperature to be stabilized within 4 degree.

Also it needs to be a portable design, assuming it will be connected with an external power supply. 12V is assumed for voltage input.

System Overview

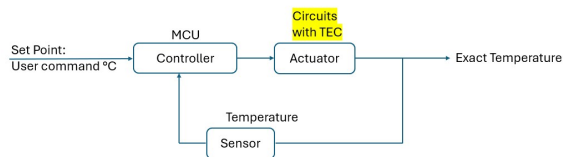
The system should be designed to accept the user's command input of intended temperature (4 degrees) to drive the TEC load to regulate the sample temperature.

In the ideal situation, a fixed input should just be able to achieve the target temperature in the system. However, in practice, the system should face a lot of disturbance in a dynamic environment like ambient temperature change, so the control system should not be designed as an open-loop one. We need to introduce feedback into the system to enable continuous adjustment during operation.

Therefore, it is proposed that a closed-loop control is required in our system. typically a closed-loop control contains several elements and their corresponding target in the project would:

- Plant: temperature
- Set point: User command input for temperature
- Feedback signal: electrical signal V that mapped from temperature
- Sensor: a temperature sensor that convert temperature to Voltage
- Error: Set point - Feedback signal that will be processed inside controller MCU to define the input for actuator!
- Controller: An MCU will be used to accept user's input command and process with the feedback signal
- actuator: to be discussed later for how actually the controller to be controlled the TEC to maintain the plant temperature

A closed-loop control diagram:

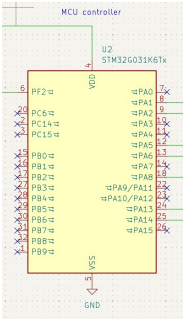


Circuit Design and Device Selection

controller selection

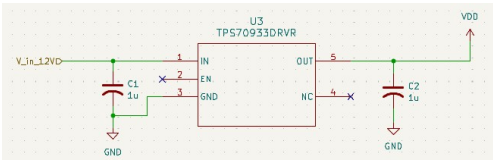
selected MCU: <https://www.st.com/resource/en/datasheet/stm32g031c6.pdf>
stm32g031c6
This MCU is selected as the main controller for the TEC temperature-control system.

The MCU will accept the user temperature setpoint input and processes it together with the feedback temperature from the sensor to generate control signal for the TEC driver circuit.



power supply for MCU 3.3V from 12V

using LDO for converting 12V into 3.3V power supply for MCU:



discussion of the actuator

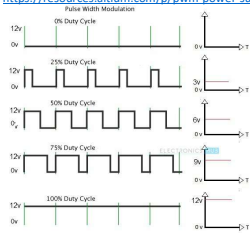
The actuator should receive a control signal from the MCU controller side and converts it to the electrical power input to the TEC to drive the temperature. This will be a digital to analog signal process.

The first thought in my head was using an external DAC, but this would add a lot of cost and complexity. Since most MCUs can generate PWM (Pulse Width Modulation) signals natively, PWM provides a more practical solution for mapping the digital input to analog signal for device control.

PWM is a switching signal that alternates between high and low voltage level with a controllable duty cycle. We can use this control signal to control the enable port for a power supply stage, enabling or disabling the connection between power supply and the load.

Intuitively, a switching input will bring the switching output. It seems we would expect the output to be another High voltage PWM, but actually we can use a filter to smooth the output to perform as the averaging voltage output, thus realizing the goal of control the behavior of PWM signal (the duty cycle) to control the voltage level to drive the TEC for different effect.

By controlling the duty cycle of the PWM signal, we can control the power stage voltage level to the TEC input
<https://resources.altium.com/p/pwm-power-supply-design>



So a sub control system become clear:
PWM input -> power supply stage -> drive the TEC

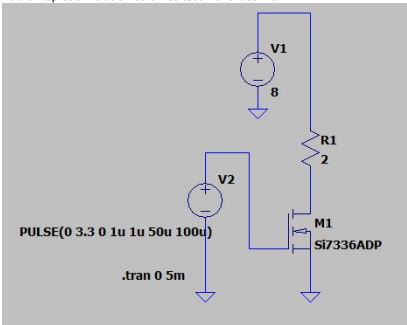
Thoughts for the power stage

I first think about if I could use simple circuit to design that

1. Low side switch
PWM signal to control the NMOS as a low side switch to control power through the TEC device
By controlling the duty cycle of the PWM, we can control the duty cycle of the current through the TEC device, so that the average current and average power delivered to the TEC can be controlled. A lower duty cycle means the TEC receives power for a smaller fraction of each switching period, less cooling effect, so the control is linked together.

For simulation purposes, since TEC spec is V 0-8V / A 0-4A, I assume the equivalent resistance is 2 Ω.

I did a Ltspice simulation below to test if this idea work.



Green:
NMOS V_Drain





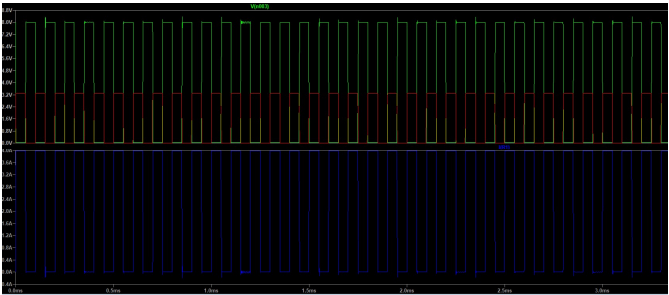
Green:
NMOS V_{Drain}

Red:
NMOS V_{Gate}

So the NMOS is working as a switch

Blue:
the TEC current

this also turns on and off.
Therefore, the duty cycle controls the average current and average power delivered to the TEC



However, this approach is not practical as the current keep switching between its peak value and 0 A, this will not allow for precise and smooth control as such switching will create a lot of ripper in cooling effect (We will be doing spike cooling!) , and the spike in current and voltage will create EMI effect for the circuit that the circuit and MCU may lose stability.

We may be able to reach a good range of temperature control by using this method (do a sort of PWM control in the TEC current, but this will not suit for the purpose for hospital sample transfer usage, which need high precision and smooth control.

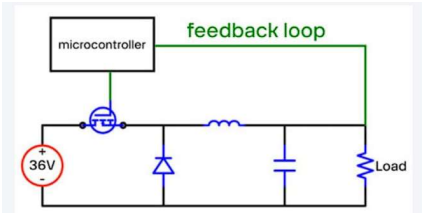
2 PWM controlled DC-DC buck converter
Inspired from the first exploration above using mosfet, for a reliable temperature control system, we may want the change to be smooth, so a smooth change of current and voltage is expected.

Also, don't forget our design need to be portable. 12V seems to be a common input supply. I assume here the portable power input voltage to be 12V.

A design goal to be:
input: 12V input, PWM control signal
output: controlled smooth continuous TEC output

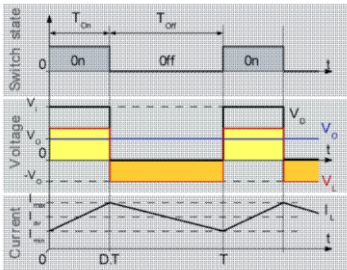
So a buck converter is for this case.
find a good article to introduce the concept <https://www.digikey.com/en/maker/tutorials/2024/how-do-buck-converters-work>

A DC-DC converter could be useful for smooth voltage and current output, while the input remain PWM. DC-DC converter is a kind of power electronics, which is built from inductor, capacitor, and have inside feedback loop to regulate the output ripper.



Change the low-side switch to high-side switch for averaging effect, and an LC output filter smooths the switching waveform.

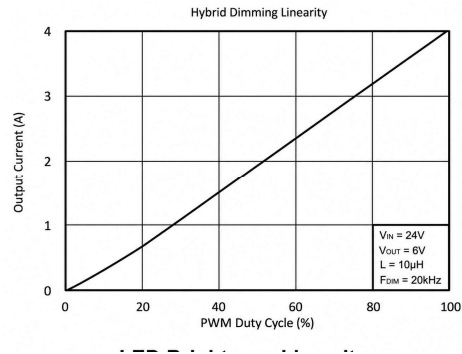
https://en.wikipedia.org/wiki/Buck_converter

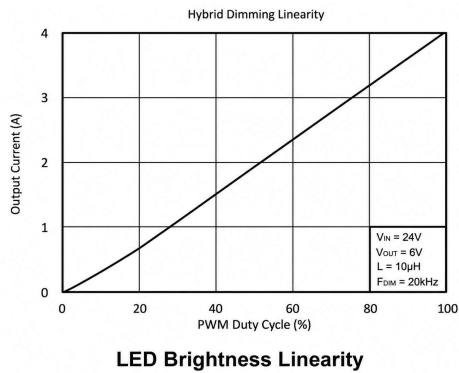


This allows the TEC to receive a lower-ripple and more stable V/I output instead of large current spike.

Thankfully there has been many options for ICs for buck converter. And also IC design would be much better as they have been tested pretty well.

a good IC I found which apply buck converter ideas to achieve mapping between PWM duty cycle and output current level to be: <https://www.digikey.com.au/en/products/detail/texas-instruments/TPS922055DRRR/22106744>





This IC can map the PWM duty cycle from 0% to 100% to an output current range of 0 A to 4 A, if the circuit configuration is set up properly. This IC chip TPS922055 acts as a linear current regulator! PWM duty cycle is in linear relationship with Output current.

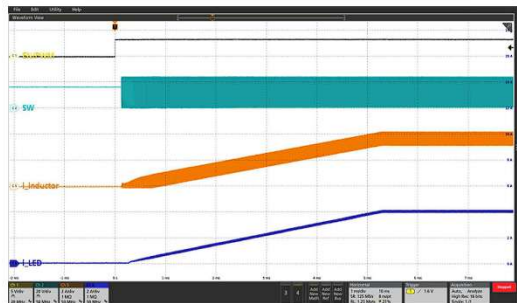


Figure 8-6. Start-Up at $PWM_{ADIM} = 100\%$, 1kHz

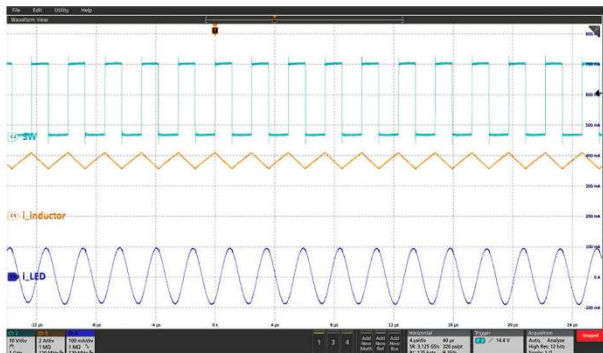


Figure 8-2. LED Current Ripple at $PWM_{ADIM} = 100\%$, 1kHz and $F_{SW} = 400kHz$

ripple to be within 100mA; compare with the first exploration of spike change...

so from above, the second one using buck converter is much more realistic

chip configuration
 following the typical application circuit in the datasheet:

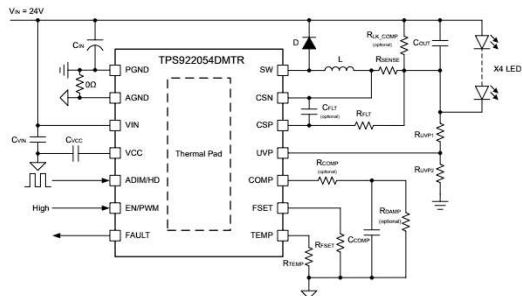


Figure 8-1. 24V Input, 4A Output, 4-piece WLED, Analog Dimming Reference Design

<https://www.ti.com/lit/ds/symlink/tps922052.pdf>

input configuration

1. mode selection: ADIM/HD and EN/PWM
according to the datasheet table 7-2, and description, I should setup for analog dimming mode to enable the chip for PWM control
simple high/low sequencing signals at startup. In PWM dimming mode, once the dimming mode is configured, LED is turned on and off corresponding to on and off of the PWM input signal at PWM input pin. The PWM dimming mode supports ultra-narrow pulse width down to 150ns. In analog dimming mode, LED current is regulated corresponding to the pulse width duty cycle of the PWM input signal at ADIM input pin. In hybrid dimming mode, the LED current is controlled by a predetermined combination of analog dimming and PWM dimming through the PWM input signal at PWM input pin. In flexible dimming mode, the LED current is controlled by analog dimming through the PWM input signal at ADIM input pins and PWM dimming through the PWM input

Table 7-2. Dimming Mode Configuration		
Dimming Mode	EN/PWM Pin	ADIM/HD Pin
PWM Dimming	PWM signal	High
Analog Dimming	High	PWM signal
Hybrid Dimming	PWM signal	Low
Flexible Dimming	PWM signal	PWM signal

For MCU pin to output PWM signal, select the pin that can output TIMER, here I select PA7 for outputting PWM

Advanced-control timer (TIM1)

The advanced-control timer can be seen as a three-phase PWM unit multiplexed on 6 channels. It has complementary PWM outputs with programmable inserted dead-times. It can also be used as a general purpose timer. The four independent channels can from datasheet pin assignment

PA8	I/O	FT	-	MCO, SPI2_NSS, TIM1_CH1, LPTIM2_OUT, EVENTOUT
-----	-----	----	---	---

Also use PA5, PA6 GPIO for device EN pin and Fault pin connection to set the EN pin active high, and catch potential fault case from the device

PA8 -> PWM signal; PA5 -> EN (active high); PA6 -> FAULT (Fault Detection)

2. FSET

configuration is set by R_FSET
this resistor is to define the system switching frequency.
for good system efficiency and good thermal behavior, choose switching freq to be 600kHz, according to datasheet, R_FSET to be 59 k Ω.

3. inductor selection

following design procedure from section 8 in datasheet

$$L = \frac{V_{OUT} \times (V_{IN(max)} - V_{OUT})}{V_{IN(max)} \times K_{IND} \times I_{L(max)} \times f_{SW}} \tag{2}$$

- where
- K_{IND} is a coefficient that represents the amount of inductor ripple current relative to the maximum LED current.
 - I_{L(max)} is the maximum inductor current.
 - f_{sw} is the switching frequency.
 - V_{IN(max)} is the maximum input voltage.
 - V_{OUT} is the sum of the voltage across LED load and the voltage across sense resistor.

The output current and voltage ripple may cause damage or stress on the TEC device, if the TEC device rating current and voltage is 4A, 8V strictly. The excessive ripple peak will exceed the rating current and voltage to cause device safety concerns. More information is needed for TEC to confirm if we need a margin design to set the designed output lower than the max rated value for protection.

Also from the datasheet equation (2), the amount of inductor ripple is inversely proportional to the inductance value L, so reducing the ripple will also require a larger inductor. This will also be a design tradeoff to be consider between size and ripple performance.

Further prototype testing is needed to verify how the mapping between ripple amounts and inductance value in real circuit setup and to confirm if a low ripple and a margin design is needed for protection of the TEC device.

For design here I assume the rating current and voltage to be 4A and 8V. So to ensure the TEC operates safely, the peak current should be smaller than 4 A.
for designed ripple of current, assuming a 10% peak-to-peak inductor current ripple for safety margin.

Calculate the allowable target average current in reverse from I_peak:

from datasheet:

The peak-inductor-current and RMS current equations are shown in Equation 4 and Equation 5.

$$I_{L(peak)} = I_{L(max)} + \frac{I_{L(ripple)}}{2} \tag{4}$$

$$4\text{ A} = I_{L(max)} + \frac{I_{L(max)} \times 0.1}{2}$$

$$\text{so } I_{L(max)} = 3.81\text{ A}$$

corresponding V_{out} should be 3.81 A x 2 Ω = 7.82 V

so in our design actually:

- V_{out}: 7.82 V
- I_{max}: 3.81 A
- K_{IND}: 0.1
- V_{in_max}: 12V
- f_{sw}: from above we set for 600kHz

$$\text{so } L = \frac{7.82\text{ V} \times (12\text{ V} - 7.82\text{ V})}{12\text{ V} \times 0.1 \times 3.81\text{ A} \times 600\text{ kHz}} = 11.91\text{ }\mu\text{H} \approx 10\text{ }\mu\text{H (for standard inductance value)}$$

come back to the equation again for exact I_{L(ripple)}

$$L = \frac{V_{OUT} \times (V_{IN(max)} - V_{OUT})}{V_{IN(max)} \times K_{IND} \times I_{L(max)} \times f_{SW}}$$

$$I_{L(ripple)} = 0.454\text{ A}$$

4. input capacitor selection

8.2.1.2.2 Input Capacitor Selection

An input capacitor is required to reduce the surge current drawn from the input supply and the switching noise coming from the device. Electrolytic capacitors are recommended for energy storage. Ceramic capacitors with X5R or X7R dielectrics are highly recommended because of their low ESR and small temperature coefficients.

from datasheet recommended value:

C_{in} = a 68μF, 50V electrolytic capacitor, a 22μF, 50V X7R ceramic capacitor
C_{vin} = 0.1μF, 50V X7R ceramic capacitor
C_{vcc} = 1μF, 16V, X7R

5. Output capacitor selection

Also we will have the output capacitor to further the ripple at output side for more protection of TEC device
equations given by datasheet for calculation such:

See Equation 7, Equation 8, and Equation 9.

$$R_{LED} = \frac{\Delta V_F}{\Delta I_F} \times \# \text{ of LEDs} \tag{7}$$

$$Z_{COUT} = \frac{R_{LED} \times I_{LED(ripple)}}{I_{L(ripple)} - I_{LED(ripple)}} \tag{8}$$

$$C_{COUT} = \frac{1}{2\pi \times f_{SW} \times Z_{COUT}} \tag{9}$$

in our case for TEC, $R_{TEC} = 2 \, \Omega$
And we may want to further decrease the $I_{TEC(ripple)}$ to 100 mA for good stability
 $Z_{out} = \frac{2 \, V \times 0.1 \, A}{0.454 \, A - 0.1 \, A} = 0.565 \, \Omega$
 $C_{out} = \frac{1}{2\pi \times 600 \, \text{kHz} \times 0.565} = 0.47 \, \mu\text{H}$
for guarantee filtering, a $0.47 \, \mu\text{F}$, 16V ceramic capacitor in parallel with a $0.1 \, \mu\text{F}$ 16V ceramic capacitor is chosen for designed ripple of 100mA

From above, $I_{TEC(ripple)}$ is controlled to be within 100mA,
 $I_{TEC(peak)}$ will be 3.91 A if all principles follow well, giving $1 - \frac{3.91 \, A}{4 \, A} = 2.25 \, \%$ safety margin
But surely the real setup should have some bias, further prototyping test to define the final value is needed after this initial design.

6. CSN (current sensing negative) and CSP (current sensing positive) for R_sense

This is for IC inner feedback control setup for stabilizing the current, the configuration is set up from R_sense and this will define the max scale current we would like to drive for our device according to the datasheet, R_sense to be 200mV/I_drive_max = 200mV/3.81 A = 52.5 mΩ

7. Undervoltage detection pin

UVP	11	11	9	I	Undervoltage detection pin. Add different resistor dividers to set the LED open detection thresholds. Leave the pin floating if undervoltage threshold is zero.
-----	----	----	---	---	---

choose to float the pin for now

8. Schottky diode

SW	14	14	12	P	Switching node pin. Internally connected to the low-side MOSFET. Connect with the power inductor and the Schottky diode.
----	----	----	----	---	--

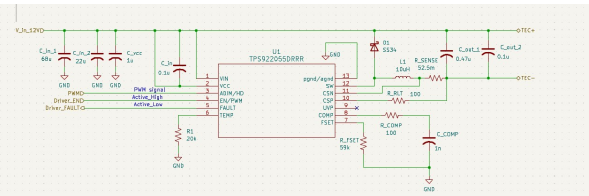
9. Other components

from datasheet suggestions set value for :
8.2.2.2.3 Other External Components Selection
In this design, a 100Ω, 0603 resistor is recommended for R_{CLT} at CSP pin to avoid noise injection and increase robustness. An optional 1nF, 50V X7R ceramic capacitor is chosen for C_{FLT} across CSP-CSN pins to filter high-frequency noise of sense feedback.
For loop stability, it is recommended to select a 1nF, 10V X7R ceramic capacitor for C_{COMP} and an optional 100Ω resistor for R_{COMP} . An optional 1MΩ resistor is chosen for R_{DAMP} to suppress the overshoot current at rising edge of PWM on. An optional resistor is chosen for R_{X_COMP} to compensate the common-node leakage current and avoid it passing through LEDs.

THERMAL PROTECTION			
Ts	Thermal foldback starting temperature threshold	$R_{sense} = 200\Omega$	130 °C

Choose to remain necessary components to reduce cost

So the circuit for actuator after discussion is:



From above, the actuator equation to be:
 $I_{TEC} = 3.81 \times D$
where D is the PWM Duty Cycle from the MCU, $0 < D < 1$.

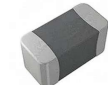
discussion of the feedback sensors

In the description, we have 2 redundant NTC as the feedback temperature for close loop control

There are many different type of NTC resistors to select in Digi-key:

1. SMD (Surface Mount Device) type NTC [not selected]

<https://www.digikey.com/en/products/detail/tdk/NTCG203FH332J1/5285713>



This type is cheapest, (\$0.22 for one) However, SMD type will not be ideal for our application as SMD resistor will need to be directly soldered on the PCB, and the components on the PCB will dissipate extra heats during operation, resulting in bias in reading temperature.

2. pearl-shaped Leaded NTC[selected]

<https://www.digikey.com/en/products/detail/semitec-usa-corp/103AT-4-70374/16578953>



This type is budget friendly (\$0.68 for one), and, This leaded type of sensors is great for our applications as it can be used as a flying lead probe attached to the TEC plate away from the PCB board, providing focused reading temperatures.

according to the spec, precision looks great with 1% tolerance:

Specifications

Product number	R ₂₅ ¹	R ₂₅ tolerance	B value ²	Dissipation factor (mW / °C)	Thermal time constant (s) ³	Rated power at 25 °C (mW)	Operating temperature range (°C)
102AT-11	1.00 kΩ	± 1%	3100 K ± 1%	approx. 2.6	approx. 75	13	- 50 to 90
202AT-11	2.00 kΩ		3182 K ± 1%				
502AT-11	5.00 kΩ		3324 K ± 1%				
103AT-11	10.0 kΩ		3435 K ± 1%				
203AT-11	20.0 kΩ		4013 K ± 1%				
102AT-2	1.00 kΩ		3100 K ± 1%	approx. 2.0	approx. 15	10	- 50 to 90
202AT-2	2.00 kΩ		3182 K ± 1%				
502AT-2	5.00 kΩ		3324 K ± 1%				
103AT-2-3	10.0 kΩ		3435 K ± 1%				
203AT-2	20.0 kΩ		4013 K ± 1%				
103AT-4	10.0 kΩ		3435 K ± 1%	approx. 10	approx. 10		- 30 to 90
103AT-5	10.0 kΩ		3435 K ± 1%	approx. 2.5	approx. 15	12.5	- 50 to 110

¹ Rated zero-power resistance at 25 °C ± 1 °C. B value calculated from rated zero-power resistance at 25 °C and 85 °C.

² Time required to reach 63.2% of temperature difference. Measured with sensor suspended in still air.

The selected NTC is 103AT-2, according to the sheet above,

- the resistance of the thermistor at temperature 25 degree is 10k = 10000 Ω
- B value: 3435K

Relationship between R and Temperature

But I am confused how can I read the real temperature from the NTC resistor, noticing the B value² is listed,

search in wiki, I found corresponding equation for R-Temp mapping:

<https://en.wikipedia.org/wiki/Thermistor>

B or β parameter equation

NTC thermistors can also be characterised with the B (or β) parameter equation, which is essentially the **Steinhart-Hart equation** with $a = 1/T_0 - (1/B) \ln R_0$, $b = 1/B$ and $c = 0$,

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \ln \frac{R}{R_0},$$

where the temperatures and the B parameter are in **kelvins**, and R₀ is the resistance of the thermistor at temperature T₀ (25 °C = 298.15 K).^[15] Solving for R yields

$$R = R_0 e^{B \left(\frac{1}{T} - \frac{1}{T_0} \right)}$$

or, alternatively,

$$R = r_{\infty} e^{B/T},$$

where $r_{\infty} = R_0 e^{-B/T_0}$.

This can be solved for the temperature:

$$T = \frac{B}{\ln(R/r_{\infty})} = \frac{T_0 * B}{T_0 * \ln(R/R_0) + B}.$$

The B-parameter equation can also be written as $\ln R = B/T + \ln r_{\infty}$. This can be used to convert the function of resistance vs. temperature of a thermistor into a linear function of $\ln R$ vs. $1/T$. The average slope of this function will then yield an estimate of the value of the B parameter.

so from above equation and parameter reading from the datasheet of the selected NTC product:

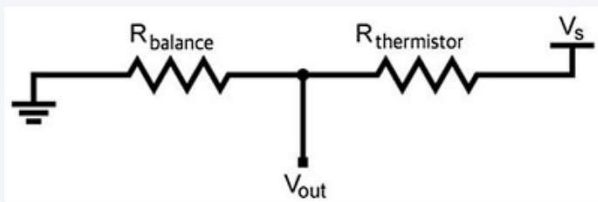
$$T_{\text{measured}} = \frac{298.15 \text{ K} * 3435 \text{ K}}{298.15 \text{ K} * \ln \left(\frac{R}{10000} \right) + 3435} \text{ and } R \text{ is the only variable in this equation!}$$

Resistance reading circuit

Reading from <https://www.digikey.com/en/maker/projects/how-to-measure-temperature-with-an-ntc-thermistor/4a4b326095f144029d7f72eca589ca54>

Measuring Resistance with an Arduino

Since we've found the correct approach, we need to now figure out how to actually measure the resistance with our Arduino before we can plug it into the β equation. This can be done using a voltage divider:



we can use a voltage divider to read the resistance value

Generally, we use a voltage divider with the equation below:

$$V_{\text{out}} = V_s \cdot (R_{\text{balance}} / (R_{\text{thermistor}} + R_{\text{balance}}))$$

However, we do not want V_{out} as the answer - we want R_{thermistor}. We'll solve for that with:

$$R_{\text{thermistor}} = R_{\text{balance}} \cdot (V_s / V_{\text{out}} - 1)$$

We're about to perfect, but we need to measure our voltage output as well as the supply voltage. This is where the Arduino's built-in ADC comes in handy.

We can represent the voltage as a digital number within a certain scale. So, our equation winds up as:

$$R_{\text{thermistor}} = R_{\text{balance}} \cdot (D_{\text{max}} / D_{\text{measured}} - 1)$$

V_{out} should be connected with MCU ADC pin to read the voltage.

The MCU we used is STM32G031K6T6. According to the datasheet <https://www.st.com/resource/en/datasheet/stm32g031c6.pdf>

3.14 Analog-to-digital converter (ADC)

A native **12-bit analog-to-digital converter** is embedded into STM32G031x4/x6/x8 devices.

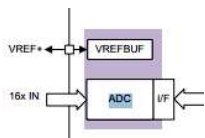


Table 59. ADC characteristics⁽¹⁾

Symbol	Parameter	Conditions ⁽²⁾	Min	Typ	Max	Unit
V _{DDA}	Analog supply voltage	-	1.62	-	3.8	V
V _{REF+}	Positive reference voltage	V _{DDA} + 2 V	2	-	V _{DDA}	V
		V _{DDA} + 2 V			V _{DDA}	
		Reverse 1	0.14	-	16	

Table 18. Voltage characteristics				
Symbol	Settings	Min	Max	Unit
V _{DD}	External supply voltage	-0.3	4.0	V ¹
V _{BAT}	External supply voltage on VBAT pin	-0.3	4.0	V
V _{REF+}	External voltage on VREF+ pin	-0.3	Min(V _{DD} + 0.4, 4.0)	V
V _{FT_XX}	Input voltage on FT_XX	-0.3	V _{DD} + 4.0 ⁽²⁾⁽³⁾	V
V _{FT_XX}	Input voltage on any other pin	-0.3	4.0	V

1. Refer to Table 19 for the maximum allowed injected current values.
2. To sustain a voltage higher than 4 V the internal pull-up/down resistors must be disabled.
3. When an FT_XX pin is used by an analog peripheral such as ADC, the maximum V_{FT_XX} is 4 V.

12-bit ADC give $ADC_max = 4095$, and its corresponding voltage is $V_REF+ = V_DDA$, which will be connected with 3.3V with V_DD in my design
Therefore, using $R_balance = 10k \Omega$ and the selected MCU, the feedback measuring equation to be:

$$R_{measured} = 10k \Omega (4095/ADC_{measured} - 1)$$

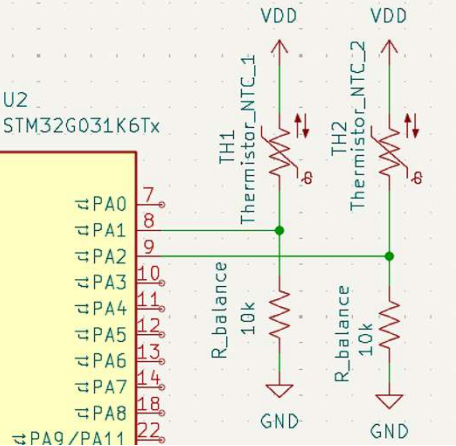
$$T_{measured} = \frac{298.15K * 3435K}{298.15K * \ln\left(\frac{R_{measured}}{10000}\right) + 3435}$$

The reason to use two NTC

The challenge description asks to use two redundant NTC thermistors.
one NTC should be used for sensing the feedback measured temperature,
another NTC i could be useful for sensing the environmental temperature outside the cooling box, this will help to describe the Thermodynamic module.

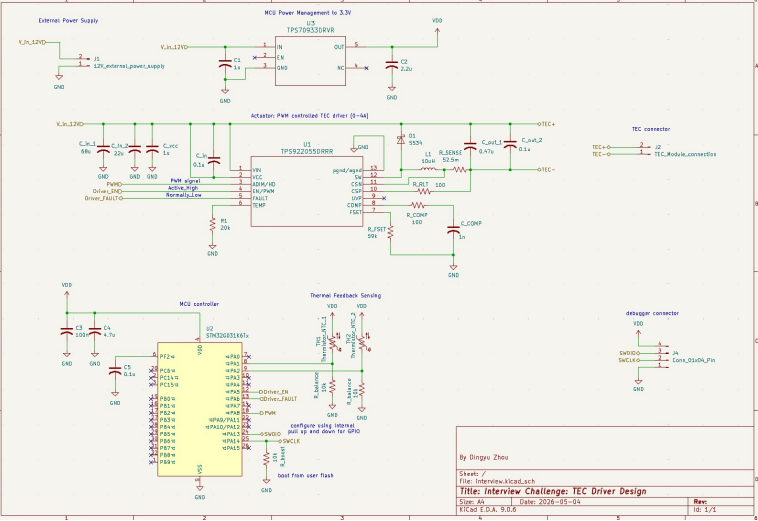
PA1	I/O	FT_ea	(3)	SPI1_SCK I2S1_OK, USART2_RTS_DE_CK, TIM2_CH2_I2C1_SMBA EVENTOUT	ADC_IN1
PA2	I/O	FT_a	(3)	SPI1_MOSI I2S1_SD, USART2_TX, TIM2_CH3, LPUART1_TX	ADC_IN2, WKUP4_LSCO

ch1 and ch2 at on board MCU ADC, the circuit for reading
Therefore, utilizing ch1 and ch2 at on board MCU ADC, the circuit for reading the NTC sensors around is as follows:



Hardware implementation

In conclusion of the circuit system selection discussed above, and with supporting decoupling capacitors added for the MCU, as well as connections for debugging and flashing pins, the overall hardware implementation schematic is proposed:



The schematic is divided into several functional modules.

- External Power and Connectors

The external power connector provides the 12V input supply for the system. The TEC connector is used to connect the driver output to the TEC device. A debugging connector is for MCU flashing and debugging.

- MCU Power Management

The MCU power-management module uses the TPS7B9933QDBVR LDO regulator to convert the 12V input supply into a regulated 3.3V supply for the MCU and its supporting circuitry.

- MCU Controller

The STM32G031K6Tx MCU is used as the main processor of the system. It reads the thermal feedback signals, processes the control algorithm, and generates the PWM control signal for the TEC driver.

- PWM-Controlled TEC Driver

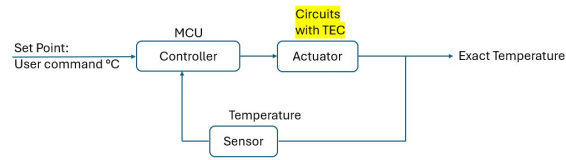
The TEC driver stage uses the TPS922055DRR driver IC as the actuator of the system. It receives the PWM control signal from the MCU and controls the TEC output current. The designed output current range is approximately 0A to 3.81A.

- Thermal Feedback Sensing

The thermal feedback sensing module uses pearl-shaped Leaded NTCs connected to the MCU ADC inputs. The MCU reads these feedback signals to estimate the sample temperature and environmental temperature, and adjust the PWM command for closed-loop temperature control.

Control Analysis and test

So far, we have designed the hardware/circuit part of the control system. This brings up the equations needed for the software control and analysis. In the following sections, I will discuss the different parts of equations, including the actuator equation, thermal feedback sensing equation, PI control equation, and thermodynamic equation. After deriving these equations, they will be connected together to form a closed-loop control system model. Then the complete control loop should be tested using Python simulation in s-domain and simulate in time domain to verify whether the system is stable, converges toward the target temperature, and behaves as expected.



control equation

1. the actuator equation:

from hardware part we have derived:
 $I_{TEC} = 3.81 * D_{PWM}$

2. For the feedback equation:

$$R_{measured} = 10\text{ k} (4095/D_{measured} - 1)$$

$$T_{measured} = \frac{298.15 + 3435}{298.15 + \ln\left(\frac{R_{measured}}{10000}\right) + 3435}$$

3. Controller equation

The controller will be used to process feedback and set point temperature input for the required D_{PWM} value, the control target is to reduce the temperature error:

$$e(t) = T_{measured}(t) - T_{set}$$

For this implementation, I would like to try PI controller:

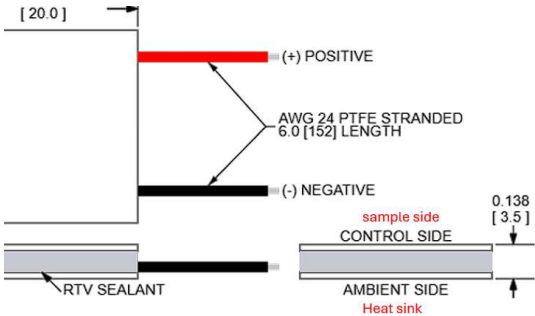
$$D_{PWM}(t) = K_p e(t) + K_i \int e(t) dt;$$

$$0 < D_{PWM}(t) < 1$$

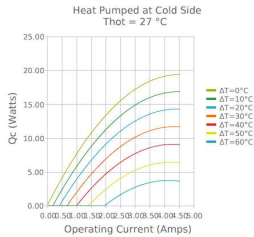
Intuitively, if measured temperature is larger than set point temperature, $e(t) > 0$, Then we want the TEC device to increase cooling effect, so the D_{PWM} should increase.

4. Output Thermodynamic equation I_{TEC} to ΔT for target temperature

Reading through the referenced TEC datasheet, the TEC thermal behavior cannot be described by a simple linear equation from I_{TEC} to temperature.



We need to use an external heat sink to help stabilize the cooling effect. For here I assume the heat sink used can keep maintaining 27 degree behind the ambient side so that the datasheet module could be valid. The cooling capacity depends on both the TEC operating current and the temperature difference between the hot side(ambient side: heat sink) and the cold side (sample side).



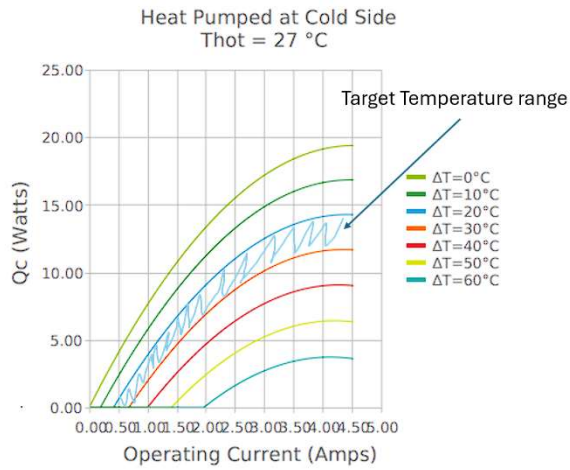
Datasheet describe cooling effect as Q_c , which is the heat pumped away from the cold side of the TEC, so we have equation of Q_c :

$$Q_c = Q_c(I_{TEC}, \Delta T)$$

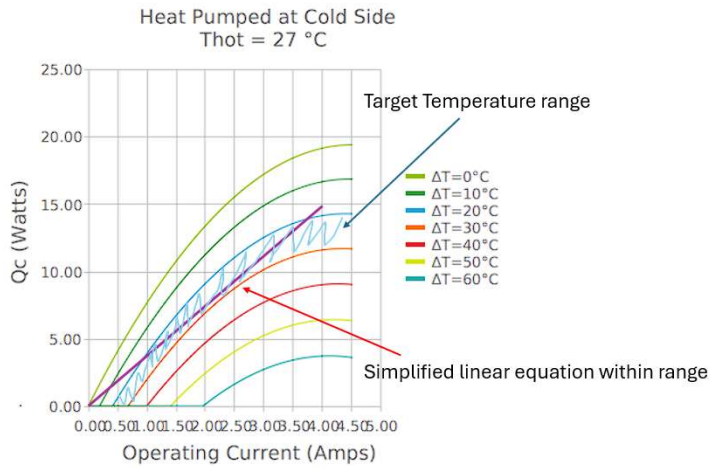
where:
 I_{TEC} is the operating current in the TEC,
 $\Delta T = T_{heatsink} - T_{sample}$

But the datasheet doesn't describe a direct equation from I to Q_c , it just provides with performance curve under different ΔT . For the preliminary control model, I found it is not necessary to use the full nonlinear datasheet map and consider jumping around different curve as T is changing for different ΔT . The system can be simplified by separating the control into two operating modes:

First, our target temperature T_{sample_target} is 4, so the $\Delta T_goal = T_{heatsink} - T_{sample_target}$ should be 23,



It should be accurate enough to simplify the range curve into one linear line:



The simplified curve equation to be:

$$Q_c(I_{REC}, \Delta T) = 3.5 I_{REC}$$

#0 shutdown mode

if the measured sample temperature has below 4 degree, then

$$D_{PWM}(t) = 0, I_{REC} = 0$$

We just let the whole TEC shutdown as cooling is meaningless here

#1 full load operating mode

Supposing initially the sample temperature is far above the target temperature of 4, We should drive the TEC at full load to quickly cool down to the target range for precise control:

define T_{band} = 1 degree

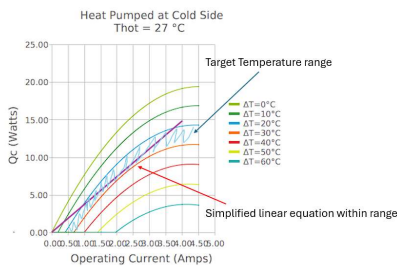
when T_{sample,measured} - T_{sample,target} > T_{band}, meaning the current sample temperature has not reached the target temperature range, push it hard!

$$D_{PWM}(t) = 1$$

$$I_{REC} = 3.81 \text{ A}$$

#2 precise control mode

After entering the target temperature range, T_{sample,measured} - T_{sample,target} < T_{band}, we start for our precise control according to curve.



$$Q_c(I_{REC}, \Delta T) = 3.5 I_{REC} \text{ become valid}$$

The reason a TEC is needed is that the sample chamber continuously receives heat leakage from the surrounding environment. Here I model the heat leakage in a simple equation:

$$Q_{leak} = K_{leak} (T_{env} - T_{sample})$$

where K_{leak} is a simplified coefficient of the environmental factor.

Then apply the heat transfer equation for the sample target: <https://adgefficiency.com/blog/q-m-cp-dt/>

$$C_{th} \frac{dT_{sample}}{dt} = Q_{in} - Q_{out} = Q_{leak} - Q_c = K_{leak} (T_{env} - T_{sample}) - Q_c(I_{REC}, \Delta T); C_{th} = \text{cm, which is a sample characteristic}$$

the above equation gives the mapping from I_TEC to T_sample!

conclusion of equations and system flows:

Here summarizes the full control logic and the equations used in the simplified TEC temperature-control model.

The system should receive the target temperature setpoint, reads the feedback temperature from the sample-side NTC sensor, and reads the environment NTC sensor to estimate the heat leakage from the surrounding environment. The heatsink temperature is assumed to be constant 27 degrees :

So Given Temperature setpoint T_{set} (t); D_{measured_{env}} (t); D_{measured_{feedback}} (t); T_{heatsink} = 27

The software control logic is divided into three modes:

1. shutdown mode
- If the measured sample temperature is below the target temperature, the TEC is turned off, that is to say:
If $T_{measured_{feedback}}(t) < 4 \text{ degree}$, then keep $D_{PWM} = 0$;
2. Full-load cooling mode:
- If the measured sample temperature is far above the target range (within 1 degree), the TEC is driven at full load for fast cooldown, that is to say:
If $\Delta T = T_{heatsink} - T_{measured_{feedback}}(t) < 1$,
 $T_{measured_{feedback}} > T_{heatsink} - 1$, meaning the current temperature is way higher
then full load mode $D_{PWM} = 1$;
3. Precise PI control mode:
- Once the measured sample temperature enters the target range, the system uses PI control, that is to say:
- controller equation:

$$e(t) = T_{measured_{feedback}}(t) - T_{set}(t);$$
$$D_{PWM}(t) = K_P e(t) + K_I \int e(t) dt;$$
$$0 < D_{PWM}(t) < 1$$

- The actuator equation maps the PWM duty cycle to TEC current:

$$I_{TEC}(t) = 3.81 * D_{PWM}(t)$$

- Output Thermodynamic equation:

$R_{measured_env}(t) = 10 \text{ k} (4095/ADC_{measured_env}(t) - 1)$

$$T_{measured_env}(t) = \frac{298.15 * 3435}{298.15 * \ln\left(\frac{R_{measured_env}(t)}{10000}\right) + 3435}$$

$$C_{th} \frac{dT_{sample}(t)}{dt} = Q_{in} - Q_{out} = Q_{leak} - Q_c = K_{leak} (T_{env}(t) - T_{sample}(t)) - Q_c(I_{TEC}(t), \Delta T)$$

$$Q_c(I_{TEC}, \Delta T) = 3.5 I_{TEC}$$

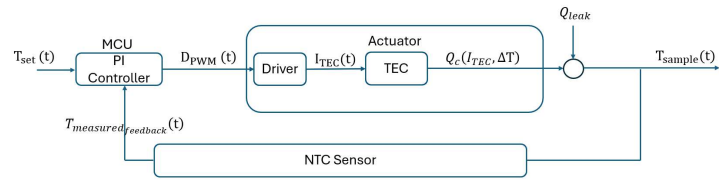
$$\Delta T = T_{heatsink} - T_{measured_{feedback}}(t)$$

- For the feedback equation:

$R_{measured_feedback}(t) = 10 \text{ k} (4095/ADC_{measured_feedback}(t) - 1)$

$$T_{measured_feedback}(t) = \frac{298.15 * 3435}{298.15 * \ln\left(\frac{R_{measured_feedback}(t)}{10000}\right) + 3435}$$

T_{set}(t), T_{measured_{feedback}}(t) → temperature error e(t)



Control system analysis in s-domain

For a control system, it is important to check whether the closed-loop output is stable, whether the system can converge to the target temperature, and whether the response time is acceptable. The designed model has introduced several design parameters, such as C_{th}, K_{leak}, K_p, and K_i. By analyzing the system in the s-domain, I hope to better understand the relationship between these parameters and reduce the arbitrary tuning to a more constructive designing. The goal of this part is to combine the equations above into one closed-loop transfer function for analysis in s-domain.

To derive the s-domain transfer function, the NTC feedback equation is not included in the analysis because it contains a nonlinear logarithmic term. For this control analysis, the temperature sensing feedback is simplified as an ideal temperature feedback measurement for temperature.

Using sympy in python for representing equation and solving for the transfer equation:

...

```

import sympy as sp
from IPython.display import display, Math

s = sp.symbols("s")

# Hyperparameters
Cth, Kleak, Kp, Ki = sp.symbols("C_th K_leak K_p K_i", positive=True)

# differential variables
Tsample, Tset, Tenv = sp.symbols("T_sample T_set T_env")
E, Dpwm, Itec, Qc = sp.symbols("E D_PWM I_TEC Q_c")

# Define Error equation:
eq_error = sp.Eq(E, Tsample - Tset)

# Define PI controller in s-domain:
eq_controller = sp.Eq(Dpwm, (Kp + Ki/s) * E)

# Define Actuator equation:
eq_actuator = sp.Eq(Itec, 3.81 * Dpwm)

# Define Simplified TEC cooling equation:
eq_cooling = sp.Eq(Qc, 3.5 * Itec)

# Thermal plant equation in s-domain:
# C_th * dT_sample/dt = K_leak(T_env - T_sample) - Q_c
# Laplace transform:
# C_th * s * T_sample = K_leak(T_env - T_sample) - Q_c
eq_thermal = sp.Eq(Cth * s * Tsample, Kleak * (Tenv - Tsample) - Qc)

## Substitute equations step by step

# Substitute error into controller
Dpwm_expr = eq_controller.rhs.subs(E, eq_error.rhs)

# Substitute D_PWM into I_TEC
Itec_expr = eq_actuator.rhs.subs(Dpwm, Dpwm_expr)

# Substitute I_TEC into Q_c
Qc_expr = eq_cooling.rhs.subs(Itec, Itec_expr)

# Substitute Q_c into thermal equation
thermal_sub = eq_thermal.subs(Qc, Qc_expr)

## Solve for T_sample(s) in terms of T_set(s) and T_env(s)
solution_Tsample = sp.solve(thermal_sub, Tsample)[0]
solution_Tsample = sp.simplify(solution_Tsample)

display(Math(r"T_{sample}(s) = " + sp.latex(solution_Tsample)))

## Transfer function T_sample(s) / T_set(s)
solution_no_env = solution_Tsample.subs(Tenv, 0)

G_closed = sp.simplify(solution_no_env / Tset)

display(Math(r"\frac{T_{sample}(s)}{T_{set}(s)} = " + sp.latex(G_closed)))

```

✓ 0.0s

$$T_{sample}(s) = \frac{2667.0K_iT_{set} + 200.0K_{leak}T_{env}s + 2667.0K_pT_{set}s}{200.0C_{th}s^2 + 2667.0K_i + 200.0K_{leak}s + 2667.0K_p s}$$

$$\frac{T_{sample}(s)}{T_{set}(s)} = \frac{2667.0(K_i + K_p s)}{200.0C_{th}s^2 + 2667.0K_i + 200.0K_{leak}s + 2667.0K_p s}$$

The transfer equation has two negative poles,

```

# Get numerator and denominator from transfer function
num, den = sp.fraction(sp.factor(G_closed))

# Solve poles from denominator = 0
poles_symbolic = sp.solve(den, s)

display(Math(r"text(Symbolic poles:)" ))

for i, p in enumerate(poles_symbolic, start=1):
    display(Math(r"s_{%d} = %s" % (i, sp.latex(p))))

```

✓ 0.0s

Closed-loop denominator: $0.0749906261717285C_{th}s^2 + 1.0K_i + 0.0749906261717285K_{leak}s + 1.0K_p s$

Symbolic poles:

$$s_1 = \frac{3.33375 \cdot 10^{-15} \left(-149981252343457.0K_{leak} - 2.0 \cdot 10^{15}K_p - \sqrt{-1.19985001874766 \cdot 10^{30}C_{th}K_i + 2.24943760545117 \cdot 10^{30}K_{leak}^2 + 5.99925009373828 \cdot 10^{30}K_{leak}K_p + 4.0 \cdot 10^{30}K_p^2} \right)}{C_{th}}$$

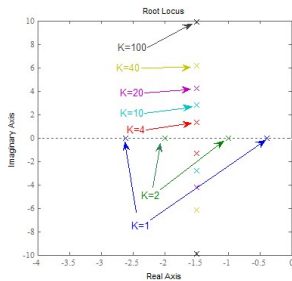
$$s_2 = \frac{3.33375 \cdot 10^{-15} \left(-149981252343457.0K_{leak} - 2.0 \cdot 10^{15}K_p + \sqrt{-1.19985001874766 \cdot 10^{30}C_{th}K_i + 2.24943760545117 \cdot 10^{30}K_{leak}^2 + 5.99925009373828 \cdot 10^{30}K_{leak}K_p + 4.0 \cdot 10^{30}K_p^2} \right)}{C_{th}}$$

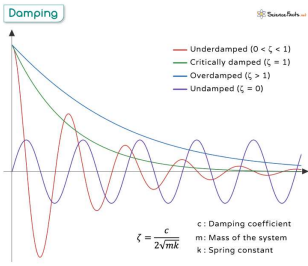
Since all denominator coefficients are positive, and the closed-loop transfer function is second order, the closed-loop poles are located in the left-half side of the s-plane.

This indicates that our control loop system is stable and the sample temperature will eventually converge towards the setpoint temperature!

hypermeter discussion

The PI controller gains K_p and K_i affect the closed-loop pole locations, which determine the response speed and damping of the temperature-control system. From my control system knowledge, poles farther to the left side from the s-plane will faster the system response time.





If the poles are real and too close to the imaginary axis, the response is slow. If the poles become complex with low damping, the response may be faster, but it will introduce overshoot or oscillation.

So for good tradeoff of response and stability, a critically damped response is preferred, which is at the point where we get two repeated real poles:
 $s_1 = s_2 = -\omega_n$

Using python to solve s and get K_p and K_i

```
num, den = sp.fraction(sp.factor(G_closed))

display(Math(r"\text{Denominator: }" + sp.latex(den)))

Denominator: 0.0749906261717285C_{th}s^2 + 1.0K_i + 0.0749906261717285K_{leak}s + 1.0K_p

# Get polynomial coefficients of denominator: a*s^2 + b*s + c
poly = sp.Poly(den, s)

a = poly.coeff_monomial(s**2)
b = poly.coeff_monomial(s)
c = poly.coeff_monomial(s**0)

display(Math(r"a = " + sp.latex(a)))
display(Math(r"b = " + sp.latex(b)))
display(Math(r"c = " + sp.latex(c)))

# Repeated pole condition: discriminant = 0
discriminant = sp.simplify(b**2 - 4*a*c)

display(Math(r"\Delta = b^2 - 4ac = " + sp.latex(discriminant)))

a = 0.0749906261717285C_{th}
b = 0.0749906261717285K_{leak} + 1.0K_p
c = 1.0K_i
\Delta = b^2 - 4ac = -0.299962504686914C_{th}K_i + (0.0749906261717285K_{leak} + 1.0K_p)^2
```

```
Ki_solution = sp.solve(sp.Eq(discriminant, 0), Ki)[0]

display(Math(r"K_i = " + sp.latex(Ki_solution)))

K_i = \frac{8.334375 \cdot 10^{-31} (149981252343457.0K_{leak} + 2.0 \cdot 10^{15}K_p)^2}{C_{th}}

B = sp.Rational(2667, 200) # 13.335 exactly, I found this value is actually the cooling gain!

den_clean = C_{th}s**2 + (K_{leak} + B*K_p)*s + B*K_i

omega_n = sp.symbols(r"\omega_n", positive=True)

desired_den = C_{th} * (s + omega_n)**2

actual_poly = sp.Poly(den_clean, s)
desired_poly = sp.Poly(sp.expand(desired_den), s)

eq_s1 = sp.Eq(actual_poly.coeff_monomial(s), desired_poly.coeff_monomial(s))
eq_s0 = sp.Eq(actual_poly.coeff_monomial(s**0), desired_poly.coeff_monomial(s**0))

solution_Kp_Ki = sp.solve([eq_s1, eq_s0], [Kp, Ki], dict=True)[0]

display(Math(r"K_p = " + sp.latex(solution_Kp_Ki[Kp])))
display(Math(r"K_i = " + sp.latex(solution_Kp_Ki[Ki])))

K_p = \frac{400C_{th}\omega_n}{2667} - \frac{200K_{leak}}{2667}
K_i = \frac{200C_{th}\omega_n^2}{2667}
```

So from the s-domain analysis above, the PI gains K_p and K_i are no longer selected as arbitrary tuning numbers. Instead, they are linked to the system hyperparameters and the desired response speed ω_n .

Now the set of hyperparameter to choose before runs:

$C_{th}, K_{leak}, \omega_n$
 where C_{th} and K_{leak} describe the simplified thermal effect in our system, and ω_n defines the desired closed-loop response speed, which users need to set them before any run.

The full control logic is tested using a time-domain simulation.

```

# Estimated parameters
Cth_val = 200.0
Kleak_val = 0.2

# cooling gain = 2667/200
cooling_gain = 13.335

# Desired settling time
Ts_val = 120.0
omega_n_val = 4 / Ts_val

# PI gains Kp and Ki
Kp_val = (2 * Cth_val * omega_n_val - Kleak_val) / cooling_gain
Ki_val = (Cth_val * omega_n_val**2) / cooling_gain

print("omega_n =", omega_n_val)
print("Kp =", Kp_val)
print("Ki =", Ki_val)

[38] ✓ 0.0s

omega_n = 0.03333333333333333
Kp = 0.9848768903887014
Ki = 0.01666458359371745

```

```

# time domain simulation of the closed-loop system with the designed PI controller
import numpy as np
import matplotlib.pyplot as plt

dt = 0.1
t_end = 600
t = np.arange(0, t_end, dt)

T_set = 4.0
T_env = 22.0
T_sample = 20.0

# Switch to PI when sample is close enough to target
T_band_high = 1.0 # start PI when T_sample <= T_set + 5°C
T_band_low = 0.0 # shut off if below target

integral_error = 0.0

T_log = []
D_log = []
I_log = []
Qc_log = []
mode_log = []

for ti in t:
    e = T_sample - T_set

    if T_sample < T_set + T_band_low:
        # Too cold: turn TEC off and reset integral to avoid windup
        D_pwm = 0.0
        integral_error = 0.0
        mode = 0 # off

    elif T_sample > T_set + T_band_high:
        # Far above target: full cooling
        D_pwm = 1.0
        mode = 1 # full drive

    else:
        # Near target: PI control
        integral_error += e * dt
        D_pwm = Kp_val * e + Ki_val * integral_error
        D_pwm = max(0.0, min(1.0, D_pwm))
        mode = 2 # PI

    I_tec = 3.81 * D_pwm
    Qc = cooling_gain * D_pwm

    Qleak = Kleak_val * (T_env - T_sample)
    dTdt = (Qleak - Qc) / Cth_val

    T_sample += dTdt * dt

    T_log.append(T_sample)
    D_log.append(D_pwm)
    I_log.append(I_tec)
    Qc_log.append(Qc)
    mode_log.append(mode)

plt.figure(figsize=(8, 4))
plt.plot(t, T_log)
plt.axhline(T_set, linestyle="--", label="Setpoint")
plt.axhline(T_set + T_band_high, linestyle=":", label="Precise control entry boundary")
plt.xlabel("Time (s)")
plt.ylabel("Sample Temperature (°C)")
plt.title("Sample Temperature Response")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(t, D_log)
plt.xlabel("Time (s)")
plt.ylabel("PWM Duty")
plt.title("PWM Command")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(t, I_log)
plt.xlabel("Time (s)")
plt.ylabel("TEC Current (A)")
plt.title("TEC Current")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(t, Qc_log)

```

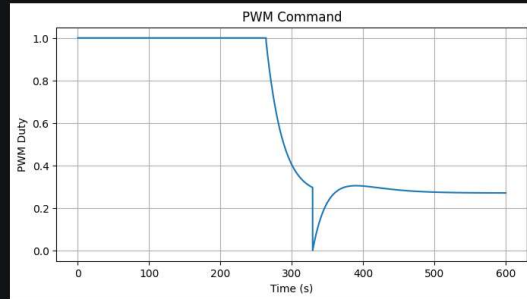
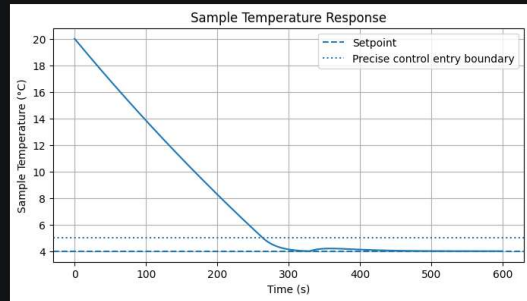
```

plt.title("TEC Current")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(t, Qc_log)
plt.xlabel("Time (s)")
plt.ylabel("Cooling Capacity Qc (W)")
plt.title("Simplified TEC Cooling Capacity")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(t, mode_log)
plt.yticks([0, 1, 2], ["Off", "Full", "PI"])
plt.xlabel("Time (s)")
plt.ylabel("Mode")
plt.title("Control Mode")
plt.grid(True)
plt.show()

```



Result to be great!

In conclusion, with the help of Python tools, I combined the control equations and derived the transfer function for the TEC control system. From the s-domain analysis, the closed-loop poles are on the left side of the s-plane, so the control system is stable and can converge to the target temperature. To get a good tradeoff between response speed and stability, I selected a critically damped response by setting two repeated real poles: $s_1 = s_2 = -\omega_n$. This gives a relationship for K_p and K_i , so they are no longer arbitrary tuning numbers. Instead, they are linked with the system parameters. So the main parameters are narrowed down to: C_{th} , R_{teak} , ω_n . Finally, I used time-domain simulation to test the control logic. The simulation shows that the system can converge to the target temperature, and the response curve looks reasonable.